

Informe Técnico Detallado

Comparación de Modelos Predictivos

GAM vs LightGBM

Predicción de Attrition (Fuga) de Clientes Bancarios

Dataset: `train_clientes.csv`

Integrantes:

Flores Quispe, Michael Sebastian

6 de agosto de 2026

Resumen Ejecutivo

Este informe documenta exhaustivamente un pipeline de *machine learning* implementado en Python/Google Colab para predecir la fuga de clientes bancarios. Se comparan dos enfoques metodológicos distintos: un **Modelo Aditivo Generalizado (GAM)** orientado a la interpretabilidad, y **LightGBM**, un gradient boosting de alto rendimiento predictivo. El análisis abarca desde la carga de datos y exploración hasta la evaluación final, incluyendo ingeniería de features temporales, balanceo de clases con SMOTE, interpretabilidad mediante valores SHAP y análisis de lift por decil.

Índice

1. Introducción y Contexto del Problema	4
1.1. Descripción del Problema	4
1.2. Objetivos del Análisis	4
1.3. Tecnologías Utilizadas	4
2. Carga y Análisis Exploratorio de Datos (EDA)	5
2.1. Carga del Dataset	5
2.2. Distribución del Target	5
2.3. Variables Temporales (Lags)	5
2.4. Valores Faltantes	5
2.5. Análisis Visual de Comportamiento Temporal	6
3. Ingeniería de Features Temporales	6

3.1.	Motivación	6
3.2.	Implementación	6
3.2.1.	Promedio y Desviación Estándar	6
3.2.2.	Tendencia Lineal	6
3.2.3.	Delta (Cambio Total)	7
4.	Preprocesamiento	7
4.1.	Imputación de Valores Faltantes	7
4.2.	Codificación de Variables Categóricas	7
4.3.	Partición Train/Test	8
4.4.	Balanceo de Clases con SMOTE	8
4.5.	Escalado de Variables	8
5.	Modelo GAM — Generalized Additive Model	9
5.1.	Fundamento Teórico	9
5.2.	Selección de Variables para el GAM	9
5.3.	Construcción y Entrenamiento	9
5.4.	Gráficos de Dependencia Parcial	10
6.	Modelo LightGBM	10
6.1.	Fundamento Teórico	10
6.2.	Configuración de Hiperparámetros	10
6.3.	Early Stopping	11
6.4.	Variables del Modelo	12
7.	Interpretabilidad de Modelos	12
7.1.	Importancia de Variables en LightGBM	12
7.2.	Valores SHAP	12
8.	Evaluación y Comparación de Modelos	13
8.1.	Curva ROC y AUC	13
8.2.	Umbral Óptimo de Youden	13
8.3.	Métricas Reportadas	13
8.4.	Validación Cruzada Estratificada	13
8.5.	Tabla Comparativa de Modelos	14
9.	Scoring Operacional	14
9.1.	Generación de Probabilidades por Cliente	14
9.2.	Análisis de Lift por Decil	14
9.3.	Archivos de Salida	15
10.	Visualizaciones Generadas	15
11.	Flujo Completo del Pipeline	15
12.	Discusión y Decisiones Metodológicas	16
12.1.	GAM: Ventajas y Limitaciones	16
12.2.	LightGBM: Ventajas y Limitaciones	17
12.3.	Sobre el Uso de SMOTE	17
12.4.	Sobre la Validación Cruzada del GAM	17

13. Conclusiones	17
A. Estructura del Dataset	18
B. Glosario	18

1. Introducción y Contexto del Problema

1.1. Descripción del Problema

El *attrition* o fuga de clientes representa uno de los principales desafíos del sector bancario. Identificar de forma anticipada a los clientes con mayor probabilidad de abandonar los servicios de la entidad permite implementar estrategias de retención focalizadas, optimizando el uso de recursos y minimizando las pérdidas de cartera.

El presente análisis utiliza un dataset de una entidad bancaria con las siguientes características generales:

Característica	Detalle
Registros totales	70,000 clientes
Variables originales	53 columnas
Variable objetivo	ATTRITION (1 = fuga, 0 = no fuga)
Tasa de fuga aproximada	15.5 %
Tipo de problema	Clasificación binaria
Horizonte temporal	6 meses de historial por variable

1.2. Objetivos del Análisis

1. Construir un pipeline completo de predicción de fuga desde la carga hasta el scoring operacional.
2. Comparar el rendimiento predictivo de GAM y LightGBM.
3. Explotar las series temporales (lags) disponibles mediante ingeniería de features.
4. Proporcionar herramientas de interpretabilidad del modelo (dependencia parcial, SHAP).
5. Generar un scoring de probabilidad de fuga por cliente como salida operacional.

1.3. Tecnologías Utilizadas

Librerías principales

- `pygam`: Implementación de Modelos Aditivos Generalizados (GAM).
- `lightgbm`: Gradient boosting optimizado para grandes datasets.
- `shap`: Valores SHAP para interpretabilidad de modelos.
- `imbalanced-learn`: SMOTE para balanceo de clases.
- `scikit-learn`: Preprocesamiento, métricas y validación cruzada.
- `pandas` / `numpy`: Manipulación y cálculo numérico.
- `matplotlib` / `seaborn`: Visualización de resultados.

2. Carga y Análisis Exploratorio de Datos (EDA)

2.1. Carga del Dataset

El archivo `train_clientes.csv` utiliza punto y coma (;) como separador decimal, característica común en datasets exportados desde sistemas europeos o latinoamericanos. La carga se realiza con:

```
1 df = pd.read_csv("train_clientes.csv", sep=";")
2 print(f"Dimensiones del dataset: {df.shape}")
3 print(f"Tasa de attrition: {df['ATTRITION'].mean()*100:.1f}%")
```

Listing 1: Carga del dataset

2.2. Distribución del Target

La variable objetivo `ATTRITION` presenta un **desbalance moderado**: aproximadamente el 15.5 % de los clientes corresponde a la clase positiva (fuga). Este desbalance, si bien no extremo, requiere manejo adecuado para evitar que los modelos optimicen únicamente la precisión sobre la clase mayoritaria (no fuga).

2.3. Variables Temporales (Lags)

Una característica distintiva del dataset es la presencia de variables con sufijos `_MENOS0` a `_MENOS5`, que representan el valor de cada indicador en los últimos 6 meses (siendo `_MENOS0` el mes más reciente y `_MENOS5` el más antiguo). Se identificaron 7 grupos de variables temporales:

Grupo	Descripción
<code>SDO_ACTIVO</code>	Saldo activo del cliente
<code>FLG_SEGURO</code>	Bandera de tenencia de seguros
<code>NRO_ACCES_CANAL1</code>	Accesos por canal 1
<code>NRO_ACCES_CANAL2</code>	Accesos por canal 2
<code>NRO_ACCES_CANAL3</code>	Accesos por canal 3
<code>NRO_ENTID_SSFF</code>	Número de entidades del sistema financiero
<code>FLG_SDO_OTSSFF</code>	Bandera de saldo en otros sistemas financieros

Cada grupo contiene 6 observaciones mensuales, sumando 42 variables de tipo lag en total.

2.4. Valores Faltantes

El análisis identifica cuatro variables con datos faltantes:

- `RANG_INGRESO` — Variable categórica ordinal
- `FLAG_LIMA_PROVINCIA` — Variable categórica binaria

- EDAD — Variable numérica
- ANTIGUEDAD — Variable numérica

2.5. Análisis Visual de Comportamiento Temporal

Para cada grupo de variables temporales se calcula y visualiza el promedio mensual diferenciado por clase (fuga vs. no fuga), permitiendo identificar patrones que sirven de señal predictiva. Por ejemplo, se espera observar en los clientes fugados una caída progresiva en el saldo activo o en los accesos a canales digitales en los meses previos a la fuga.

3. Ingeniería de Features Temporales

3.1. Motivación

Los lags individuales (6 variables por grupo) son informativos pero generan multicolinealidad y dificultan la convergencia del GAM con muchas variables. Para aprovechar la información temporal de forma compacta y crear variables más interpretables, se construyen **4 features agregadas** por cada uno de los 7 grupos:

Sufijo	Feature	Interpretación
_PROM6M	Promedio 6 meses	Nivel medio de actividad
_STD6M	Desv. estándar 6 meses	Estabilidad/volatilidad
_TEND6M	Pendiente de regresión lineal	Tendencia creciente o decreciente
_DELTA6M	Diferencia mes actual — hace 6 meses	Cambio absoluto en el periodo

Esto genera $7 \times 4 = 28$ nuevas variables de resumen temporal.

3.2. Implementación

3.2.1. Promedio y Desviación Estándar

```
1 df_feat[f'{var}_PROM6M'] = df[cols].mean(axis=1)
2 df_feat[f'{var}_STD6M']  = df[cols].std(axis=1)
```

Listing 2: Cálculo de estadísticos básicos por grupo temporal

3.2.2. Tendencia Lineal

La tendencia se estima mediante regresión lineal simple sobre los 6 puntos de tiempo, usando `numpy.polyfit`. El coeficiente de primer grado (pendiente) indica si la variable sube o baja en el periodo:

Modelo de tendencia:

$$y_t = \beta_0 + \beta_1 \cdot t + \varepsilon_t, \quad t \in \{0, 1, 2, 3, 4, 5\}$$

Se extrae $\hat{\beta}_1$ como feature `_TEND6M`.

```

1 x = np.arange(6)
2 slopes = []
3 for _, row in df[cols_chron].iterrows():
4     y = row.values.astype(float)
5     if np.std(y) > 0:
6         slope = np.polyfit(x, y, 1)[0]
7     else:
8         slope = 0.0
9     slopes.append(slope)
10 df_feat[f'{var}_TEND6M'] = slopes

```

Listing 3: Cálculo de tendencia lineal mediante regresión

3.2.3. Delta (Cambio Total)

```

1 df_feat[f'{var}_DELTA6M'] = df[cols[0]] - df[cols[5]]
2 # cols[0] = MENOS0 (actual), cols[5] = MENOS5 (hace 6 meses)

```

Listing 4: Cambio absoluto entre el mes más reciente y el más antiguo

4. Preprocesamiento

4.1. Imputación de Valores Faltantes

Se aplica una estrategia diferenciada según el tipo de variable:

- **Variables categóricas** (RANG_INGRESO, FLAG_LIMA_PROVINCIA): imputación con la **moda**, preservando la categoría más frecuente.
- **Variables numéricas** (EDAD, ANTIGUEDAD): imputación con la **mediana**, más robusta que la media frente a outliers.

4.2. Codificación de Variables Categóricas

Se utiliza **Label Encoding** para las siguientes variables ordinales:

```

1 cat_cols = ['RANG_INGRESO', 'FLAG_LIMA_PROVINCIA',
2             'RANG_SDO_PASIVO_MENOSO', 'RANG_NRO_PRODUCTOS_MENOSO']
3 for col in cat_cols:
4     df_feat[col] = le.fit_transform(df_feat[col].astype(str))

```

Listing 5: Codificación de variables ordinales

Esta elección es apropiada porque estas variables tienen un orden natural implícito (e.g., rango de ingreso 01 < rango de ingreso 02).

4.3. Partición Train/Test

Se realiza una división **estratificada** 70 %/30 %:

```
1 X_train_full, X_test, y_train_full, y_test = train_test_split(  
2     X_all, y, test_size=0.30, random_state=42, stratify=y  
3 )
```

Listing 6: División estratificada del dataset

La estratificación garantiza que ambos subconjuntos mantengan la proporción de clases original ($\approx 15.5\%$ de fuga), evitando sesgos en la evaluación.

4.4. Balanceo de Clases con SMOTE

¿Qué es SMOTE?

Synthetic Minority Over-sampling Technique (SMOTE) genera ejemplos sintéticos de la clase minoritaria interpolando entre observaciones reales de esa clase en el espacio de features, en lugar de simplemente duplicar registros existentes. Esto reduce el riesgo de sobreajuste respecto al *oversampling* simple.

```
1 smote = SMOTE(random_state=42)  
2 X_train_bal, y_train_bal = smote.fit_resample(X_train_full, y_train_full  
3 )
```

Listing 7: Aplicación de SMOTE sobre el conjunto de entrenamiento

Nota crítica: SMOTE se aplica *exclusivamente* sobre el conjunto de entrenamiento. El test set conserva la distribución real de clases para una evaluación honesta del desempeño del modelo.

4.5. Escalado de Variables

```
1 scaler = StandardScaler()  
2 X_train_scaled = scaler.fit_transform(X_train_bal)  
3 X_test_scaled = scaler.transform(X_test)
```

Listing 8: Normalización con StandardScaler

El escalado es **imprescindible para el GAM**, cuyas funciones spline son sensibles a la escala. LightGBM, al estar basado en árboles de decisión, es invariante a transformaciones monótonas de las variables, por lo que no requiere escalado; sin embargo, se aplica el mismo preprocesamiento para mantener consistencia.

5. Modelo GAM — Generalized Additive Model

5.1. Fundamento Teórico

Un Modelo Aditivo Generalizado (GAM) extiende la regresión logística permitiendo efectos no lineales de cada variable mediante funciones suaves (splines). Para clasificación binaria, el modelo es:

$$\text{logit}(P(Y = 1 \mid \mathbf{x})) = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_p(x_p)$$

donde $g(\cdot) = \text{logit}(\cdot)$ es la función de enlace, y cada f_i es una función suave estimada mediante splines cúbicos.

Ventaja clave: la estructura aditiva permite interpretar el efecto individual de cada variable a través de los gráficos de dependencia parcial, manteniendo la no linealidad que la regresión logística estándar no captura.

5.2. Selección de Variables para el GAM

Justificación de la selección

El GAM con `pygam` no escala eficientemente con más de 60 variables, presentando problemas de convergencia y alto tiempo de cómputo. Se seleccionan:

- 8 variables estáticas del perfil del cliente.
- 28 features temporales agregadas (4 por cada uno de los 7 grupos).

Total: **36 variables para el GAM.**

Variables estáticas seleccionadas:

- `FLG_BANCARIZADO`: indicador de bancarización previa.
- `RANG_INGRESO`: rango de ingresos del cliente.
- `FLAG_LIMA_PROVINCIA`: ubicación geográfica.
- `EDAD`: edad del cliente.
- `ANTIGUEDAD`: antigüedad en el banco.
- `RANG_SDO_PASIVO_MENOSO`: rango de saldo pasivo actual.
- `RANG_NRO_PRODUCTOS_MENOSO`: rango de número de productos.
- `FLG_NOMINA`: indicador de nómina.

5.3. Construcción y Entrenamiento

```
1 # T r m i n o s ( i ) : s p l i n e s u a v i z a d o p a r a l a v a r i a b l e e n p o s i c i o n i
2 t e r m s _ g a m = s ( 0 )
3 f o r i i n r a n g e ( 1 , l e n ( v a r s _ g a m ) ) :
```

```

4     terms_gam += s(i)
5
6 gam_model = LogisticGAM(terms_gam, n_splines=10)
7
8 # Búsqueda del parámetro de regularización lam
9 gam_model.gridsearch(X_train_gam, y_train_bal, progress=True)

```

Listing 9: Construcción dinámica del GAM y búsqueda de hiperparámetros

Hiperparámetros del GAM:

- **n_splines=10**: número de nodos del spline cúbico por variable.
- **lam**: parámetro de regularización (penalización de la rugosidad del spline). Valores altos producen curvas más suaves (mayor sesgo, menor varianza). Optimizado mediante *grid search* interno con validación cruzada.

5.4. Gráficos de Dependencia Parcial

Los gráficos de dependencia parcial (PDP) muestran $\hat{f}_i(x_i)$ en función del rango de valores de x_i , manteniendo las demás variables en sus valores observados. La banda sombreada representa el intervalo de confianza al 95 %.

```

1 for i, var_name in enumerate(vars_gam):
2     XX = gam_model.generate_X_grid(term=i)
3     pdep, confi = gam_model.partial_dependence(term=i, X=XX, width=0.95)
4     axes[i].plot(XX[:, i], pdep, color='#E53935', linewidth=2)
5     axes[i].fill_between(XX[:, i], confi[:, 0], confi[:, 1],
6                          alpha=0.3, color='#EF9A9A')

```

Listing 10: Generación de gráficos de dependencia parcial

6. Modelo LightGBM

6.1. Fundamento Teórico

LightGBM es un framework de *gradient boosting* basado en árboles de decisión que emplea dos innovaciones técnicas principales:

- **Gradient-based One-Side Sampling (GOSS)**: mantiene todas las instancias con gradientes grandes y submuestra aleatoriamente las de gradiente pequeño.
- **Exclusive Feature Bundling (EFB)**: agrupa features mutuamente excluyentes, reduciendo la dimensionalidad sin pérdida de información.

Esto lo hace considerablemente más rápido que XGBoost en datasets medianos y grandes.

6.2. Configuración de Hiperparámetros

```

1 lgbm_model = LGBMClassifier(
2     boosting_type='gbdt',
3     n_estimators=500,
4     max_depth=6,
5     num_leaves=31,
6     learning_rate=0.05,
7     min_child_samples=30,
8     subsample=0.8,
9     colsample_bytree=0.8,
10    reg_alpha=0.1,          # regularizaci3n L1
11    reg_lambda=0.1,        # regularizaci3n L2
12    random_state=42,
13    n_jobs=-1,
14    verbose=-1
15 )

```

Listing 11: Configuraci3n del modelo LightGBM

Parámetro	Valor	Propósito
n_estimators	500	Número máximo de árboles
max_depth	6	Profundidad máxima de cada árbol
num_leaves	31	Hojas máximas (controla complejidad)
learning_rate	0.05	Tasa de aprendizaje conservadora
min_child_samples	30	Mínimo de muestras por hoja
subsample	0.8	Fracci3n de filas por árbol (bagging)
colsample_bytree	0.8	Fracci3n de features por árbol
reg_alpha	0.1	Regularizaci3n L1
reg_lambda	0.1	Regularizaci3n L2

6.3. Early Stopping

Para evitar sobreajuste y reducir tiempo de cómputo, se implementa *early stopping*: el entrenamiento se detiene si el AUC en el conjunto de validaci3n no mejora en 50 rondas consecutivas.

```

1 X_tr_lgb, X_val_lgb, y_tr_lgb, y_val_lgb = train_test_split(
2     X_train_bal, y_train_bal, test_size=0.15, random_state=42
3 )
4
5 lgbm_model.fit(
6     X_tr_lgb, y_tr_lgb,
7     eval_set=[(X_val_lgb, y_val_lgb)],
8     callbacks=[lgb.early_stopping(50, verbose=False),
9                 lgb.log_evaluation(100)]
10 )

```

Listing 12: Entrenamiento con early stopping

6.4. Variables del Modelo

A diferencia del GAM, LightGBM utiliza el conjunto **completo** de variables: las 53 originales más las 28 features temporales agregadas, sumando **81 features**. Los árboles pueden identificar automáticamente cuáles son relevantes.

7. Interpretabilidad de Modelos

7.1. Importancia de Variables en LightGBM

LightGBM provee dos métricas nativas de importancia:

- **Split**: número de veces que la variable se usa en un nodo de división.
- **Gain**: ganancia total de información acumulada en todos los splits que usan esa variable. Es más informativa y robusta.

Se visualizan las **Top 20 variables por Gain**.

7.2. Valores SHAP

Fundamento de SHAP

Los valores SHAP (*SHapley Additive exPlanations*) asignan a cada variable una contribución marginal a la predicción individual, basada en la teoría de juegos cooperativos (valores de Shapley). A diferencia de las medidas de importancia global, SHAP muestra tanto la **magnitud** como la **dirección** del efecto por cliente.

```

1 # Muestra de 3000 registros para eficiencia computacional
2 sample_idx = np.random.choice(len(X_test), size=3000, replace=False)
3 X_shap_sample = X_test.iloc[sample_idx]
4
5 explainer = shap.TreeExplainer(lgbm_model)
6 shap_values = explainer.shap_values(X_shap_sample)
7
8 # Para clasificación binaria, se toma la clase 1 (fuga)
9 if isinstance(shap_values, list):
10     shap_vals = shap_values[1]
```

Listing 13: Cálculo de valores SHAP

Gráficos generados:

- **Summary plot**: cada punto representa un cliente. La posición horizontal indica el valor SHAP (impacto en log-odds de fuga); el color refleja el valor de la variable.
- **Dependence plot**: relación entre el valor de la variable más importante y su contribución SHAP, coloreada por la variable de mayor interacción.

8. Evaluación y Comparación de Modelos

8.1. Curva ROC y AUC

La curva ROC grafica la Tasa de Verdaderos Positivos (TPR) frente a la Tasa de Falsos Positivos (FPR) para todos los umbrales posibles:

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

- AUC = 0.5: clasificador aleatorio (sin información)
- AUC = 1.0: clasificador perfecto

8.2. Umbral Óptimo de Youden

En lugar de usar un umbral fijo de 0.5, se selecciona el umbral que maximiza el **índice de Youden**:

$$J = \text{TPR} - \text{FPR} = \text{Sensibilidad} + \text{Especificidad} - 1$$

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)]$$

```
1 def umbral_youden(fpr, tpr, thresholds):
2     idx = np.argmax(tpr - fpr)
3     return thresholds[idx]
```

Listing 14: Función para calcular el umbral de Youden

8.3. Métricas Reportadas

Métrica	Definición
Precision	$TP/(TP + FP)$: de los predichos como fuga, ¿cuántos son correctos?
Recall (Sensibilidad)	$TP/(TP + FN)$: de los que fugaron, ¿cuántos detectó el modelo?
F1-Score	$2 \cdot (P \cdot R)/(P + R)$: media armónica entre precision y recall
AUC-ROC	Capacidad discriminante global

8.4. Validación Cruzada Estratificada

Se aplica validación cruzada con $k = 5$ pliegues estratificados sobre el conjunto de entrenamiento original (sin SMOTE), para estimar el desempeño real de generalización:

```

1 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
2
3 cv_lgbm = cross_val_score(
4     LGBMClassifier(n_estimators=300, ...),
5     X_train_full, y_train_full,
6     cv=skf, scoring='roc_auc', n_jobs=-1
7 )
8 print(f"LightGBM CV AUC: {cv_lgbm.mean():.4f} +/- {cv_lgbm.std():.4f}")

```

Listing 15: Validación cruzada estratificada

Nota: La validación cruzada del GAM se realiza manualmente fold por fold, re-escalando los datos en cada pliegue para evitar fuga de información del test hacia el entrenamiento.

8.5. Tabla Comparativa de Modelos

Criterio	GAM	LightGBM
Variables usadas	36 (selección)	81 (todas)
Interpretabilidad	Alta (splines)	Media (SHAP)
Velocidad de entrenamiento	Lenta	Rápida
Manejo de interacciones	No (aditivo)	Sí (árboles)
Escalabilidad	Limitada	Alta
Requiere escalado	Sí	No

9. Scoring Operacional

9.1. Generación de Probabilidades por Cliente

El resultado operacional del pipeline es una tabla con la probabilidad de fuga estimada para cada cliente del conjunto de prueba:

```

1 df_scoring['PROB_FUGA_LGBM'] = y_prob_lgbm_test
2 df_scoring['PROB_FUGA_GAM'] = y_prob_gam_test
3 df_scoring['SCORE_DECIL'] = pd.qcut(
4     df_scoring['PROB_FUGA_LGBM'], q=10,
5     labels=[f'D{i}' for i in range(1, 11)]
6 )

```

Listing 16: Generación del scoring por cliente

Los clientes se agrupan en 10 deciles según su probabilidad predicha (D1 = menor riesgo, D10 = mayor riesgo).

9.2. Análisis de Lift por Decil

La curva de lift cuantifica la ganancia del modelo frente a una selección aleatoria:

$$\text{Lift}_d = \frac{\text{Tasa de fuga en decil } d}{\text{Tasa de fuga global}}$$

Por ejemplo, un lift de 3.5 en el decil 10 significa que contactando al 10 % de clientes con mayor score, se captura el 3.5 veces más fugados que si se seleccionara aleatoriamente.

9.3. Archivos de Salida

El pipeline genera dos archivos CSV de uso operacional:

Archivo	Contenido
tabla_comparativa_modelos.csv	Métricas consolidadas de ambos modelos
scoring_clientes.csv	Probabilidad de fuga + decil por cliente

10. Visualizaciones Generadas

El pipeline produce los siguientes gráficos (guardados en formato PNG a 150 DPI):

Archivo	Contenido	Bloque
01_eda_overview.png	Distribución del target + valores faltantes	Bloque 3
02_evolucion_temporal.png	Evolución mensual por variable: fuga vs no fuga	Bloque 3
03_gam_partial_dependence.png	Dependencia parcial del GAM por variable	Bloque 11
04_lgbm_feature_importance.png	Top 20 variables más importantes (LightGBM)	Bloque 13
05_shap_summary.png	Summary plot de valores SHAP (LightGBM)	Bloque 14
06_shap_dependence.png	Dependence plot SHAP de la variable top	Bloque 14
07_roc_comparison.png	Curvas ROC comparadas: GAM vs LightGBM	Bloque 15
08_confusion_matrices.png	Matrices de confusión (umbral de Youden)	Bloque 16
09_lift_deciles.png	Curva de lift por decil (LightGBM)	Bloque 20

11. Flujo Completo del Pipeline

Bloque	Nombre	Tecnología	Salida
0	Instalación de librerías	pip	Entorno configurado
1	Importación	Python / sklearn	Módulos cargados
2	Carga de datos	pandas	DataFrame (70K × 53)
3	EDA	matplotlib / seaborn	2 gráficos
4	Feature engineering temporal	numpy	+28 variables
5	Preprocesamiento	sklearn	DataFrame limpio
6	Train/test split	sklearn	70 %/30 % estratificado
7	SMOTE	imbalanced-learn	Train balanceado
8	Escalado	sklearn	Variables normalizadas
9	Selección vars GAM	pandas	36 variables
10	Entrenamiento GAM	pygam	Modelo + AUC
11	PDP del GAM	pygam / matplotlib	Gráfico PDP
12	Entrenamiento LightGBM	lightgbm	Modelo + AUC
13	Importancia de variables	lightgbm	Top 20 features
14	Valores SHAP	shap	Summary + Dependence plot
15	Curvas ROC	sklearn	Gráfico comparativo
16	Métricas de clasificación	sklearn	Reporte + matrices
17	Validación cruzada	sklearn / pygam	AUC CV ± std
18	Tabla comparativa	pandas	CSV métricas
19	Scoring por cliente	pandas	CSV scoring
20	Análisis de lift	pandas / matplotlib	Gráfico lift

12. Discusión y Decisiones Metodológicas

12.1. GAM: Ventajas y Limitaciones

Ventajas:

- La estructura aditiva garantiza una interpretación directa: el gráfico de dependencia parcial de $f_i(x_i)$ muestra el efecto no lineal de cada variable de forma aislada.
- Útil para comunicar resultados a audiencias no técnicas (reguladores, gestión de riesgos).
- El intervalo de confianza al 95 % en los PDP cuantifica la incertidumbre.

Limitaciones:

- No captura **interacciones** entre variables (por construcción, el modelo es aditivo).
- Problemas de convergencia con muchas variables (>60) en **pygam**.
- Tiempo de entrenamiento significativamente mayor, especialmente con *gridsearch*.
- Requiere escalado previo de variables.

12.2. LightGBM: Ventajas y Limitaciones

Ventajas:

- Captura automáticamente **interacciones** entre variables.
- Puede usar todas las features disponibles sin selección manual.
- Entrenamiento rápido con *early stopping*.
- Robusto al desbalance de clases.
- Interpretabilidad complementada con SHAP.

Limitaciones:

- Interpretabilidad intrínseca menor que el GAM (caja negra parcial).
- Mayor riesgo de sobreajuste si no se regula adecuadamente.

12.3. Sobre el Uso de SMOTE

Una consideración importante es que el entrenamiento de LightGBM se realiza sobre datos balanceados con SMOTE. Existe un debate en la literatura sobre si SMOTE es siempre beneficioso para modelos de gradient boosting (que ya manejan el desbalance implícitamente), pero en este caso se aplica de forma consistente para ambos modelos y permite una comparación más justa.

12.4. Sobre la Validación Cruzada del GAM

En la validación cruzada del GAM, el `StandardScaler` se re-ajusta en cada pliegue de entrenamiento y se aplica al pliegue de validación con los parámetros del train, evitando así la **fuga de datos** (*data leakage*). Esto es una buena práctica que algunos implementaciones omiten incorrectamente.

13. Conclusiones

El pipeline implementado constituye un marco completo y metodológicamente sólido para la predicción de attrition bancario. Las principales conclusiones son:

1. **Ingeniería de features temporales:** la construcción de features agregadas (promedio, desviación estándar, tendencia lineal y delta) a partir de las series de 6 meses condensa información temporal relevante y mejora la convergencia del GAM.
2. **GAM como modelo interpretable:** proporciona una explicación visual directa de cómo cada variable afecta la probabilidad de fuga, siendo especialmente valioso en contextos regulados donde se deben justificar las decisiones del modelo.
3. **LightGBM como modelo de alto rendimiento:** captura interacciones complejas entre variables, usa el conjunto completo de features y produce predicciones

más precisas en términos de AUC, complementadas con interpretabilidad post-hoc mediante SHAP.

4. **Salida operacional:** el scoring por decil permite priorizar campañas de retención de forma eficiente. Con un lift alto en los deciles superiores, el banco puede focalizar recursos en el 10-20 % de clientes con mayor riesgo real.
5. **Recomendación práctica:** en entornos donde se requiere explicabilidad regulatoria, el GAM es preferible. Para maximizar capacidad predictiva, LightGBM es superior. Una arquitectura híbrida (usar LightGBM para scoring y GAM para explicaciones) puede ser la solución óptima.

A. Estructura del Dataset

Tipo de variable	Ejemplos
Perfil estático	EDAD, ANTIGUEDAD, RANG_INGRESO, FLG_BANCARIZADO
Geográfica	FLAG_LIMA_PROVINCIA
Productos	RANG_NRO_PRODUCTOS_MENOS0, FLG_NOMINA
Saldos temporales	SDO_ACTIVO_MENOS0 a SDO_ACTIVO_MENOS5
Canales temporales	NRO_ACCES_CANAL1_MENOS0 a _MENOS5
Sistema financiero	NRO_ENTID_SSFF_MENOS0 a _MENOS5
Seguros	FLG_SEGURO_MENOS0 a _MENOS5
Target	ATTRITION (0/1)
ID	ID_CORRELATIVO, CODMES

B. Glosario

Attrition Abandono o fuga del cliente de los servicios bancarios.

AUC-ROC Área bajo la curva ROC; mide la capacidad discriminante del modelo.

SMOTE Synthetic Minority Over-sampling Technique; técnica de sobre-muestreo sintético.

GAM Generalized Additive Model; modelo aditivo generalizado con funciones suaves.

LightGBM Light Gradient Boosting Machine; implementación eficiente de gradient boosting.

SHAP SHapley Additive exPlanations; método de interpretabilidad basado en teoría de juegos.

Lift Ganancia relativa del modelo frente a una selección aleatoria.

Umbral de Youden Umbral de clasificación que maximiza la diferencia TPR – FPR.

Spline cúbico Función polinomial por partes de grado 3, continua y con derivadas continuas.

Early stopping Detención anticipada del entrenamiento cuando el desempeño en validación no mejora.